

Dossier de Présentation des Projets

Développeur Web et Web Mobile

Candidat : Harouna Soumare

Adresse : 304 rue de Belleville, 75020 Paris

Email : harounasoumare547@gmail.com

Formation : LA PLATEFORME

Début formation : 30 juin 2025

Titre visé : DWWM — TP-01280

Document produit dans le cadre de l'examen du Titre Professionnel Développeur Web et Web Mobile

Ce dossier présente les projets réalisés au cours de ma formation de 16 mois au sein de LA PLATEFORME, dans le cadre du Titre Professionnel Développeur Web et Web Mobile (DWWM —TP-01280). Ces 16 mois ont été pour moi une période d'apprentissage intense et enrichissante. En partant de bases solides en JavaScript, j'ai progressivement construit des compétences concrètes en développement web la conception d'interfaces avec React, la création d'APIs REST avec Node.js et Express, la gestion de bases de données SQL et NoSQL, et le déploiement en production. Chaque projet présenté dans ce dossier représente une étape concrète de cette progression. Je remercie l'équipe pédagogique de LA PLATEFORME pour leur accompagnement tout au long de cette formation, ainsi que les membres du jury pour le temps consacré à l'évaluation de mon travail.

Sommaire

Dossier de Présentation des Projets.....	1
Sommaire	3
Projet 1 — Site associatif AJBF (baydjam.fr)	3
1. Présentation du projet et contexte	4
Objectifs du projet	4
Méthode de travail.....	4
2. Technologies et outils utilisés	4
Front-end.....	5
Back-end	6
3. Architecture de l'application.....	8
API REST — Principales routes	9
4. Base de données	9
Sécurité de la base de données	10
5. Fonctionnalités développées	10
Page d'accueil	10
Galerie photos	10
Page FC18 — Équipe de Football.....	11
Espace administration	11
Pages légales et conformité	11
6. Captures d'écran.....	11
Hook useSEO — Gestion du référencement.....	12
Génération automatique des miniatures vidéo	12
Protection des routes admin (ProtectedRoute)	12
Route API Express — Récupération des joueurs FC18	13
Captures d'écran du site en production	13
7. Difficultés rencontrées et solutions	15
Problème 1 — Upload de vidéos bloqué (erreur 413)	16
Problème 2 — Vidéos sans aperçu avant lecture	16

Problème 3 — Configuration environnement dev vs production	16
8. Compétences DWWM couvertes	16
Projet 2 — Plateforme de festival de films IA (MarsAI)	17
1. Présentation du projet et contexte	18
2. Technologies et outils utilisés	18
3. Architecture et organisation du back-end	20
4. Base de données	21
5. Fonctionnalités développées (partie back-end)	22
Routes et contrôleurs CRUD	23
Organisation des contrôleurs	24
6. Difficultés rencontrées et solutions	24
7. Compétences DWWM couvertes	25
Projet 3 — API [Cahier de Recettes]	26
1. Présentation du projet et contexte	27
Objectifs du projet	27
Méthode de travail	27
2. Technologies et outils utilisés	27
3. Architecture de l'application	28
Routes API principales	29
SQL vs NoSQL — différence clé	30
5. Fonctionnalités développées	30
Authentification utilisateur	30
CRUD Recettes	30
Commentaires	30
Documentation Swagger	30
6. Difficultés rencontrées et solutions	31
7. Compétences DWWM couvertes	31

Projet 1 — Site associatif AJBF (baydjam.fr)

1. Présentation du projet et contexte

L'AJBF (Association de la Jeunesse de Baediam en France) est une association loi 1901 basée à Paris, dont l'objectif est de rassembler et de valoriser la communauté de Baediam établie en France. Avant ce projet, l'association n'avait aucune présence numérique : pas de site web, pas de moyen pour communiquer ses événements au grand public, et aucun espace pour mettre en valeur ses activités sportives et culturelles.

J'ai été sollicité pour concevoir et développer intégralement leur site officiel, accessible à l'adresse baydjam.fr. Ce projet a été mené de A à Z, depuis la conception des maquettes jusqu'à la mise en production, en passant par le développement du front-end et du back-end.

Le site est actuellement en version 1 — fonctionnel et en production à l'adresse baydjam.fr. Des évolutions sont prévues pour enrichir les fonctionnalités existantes.

Objectifs du projet

- Offrir à l'association une vitrine en ligne professionnelle et moderne
- Permettre aux membres et au public de consulter les événements à venir
- Présenter l'équipe de football FC18 (joueurs, trophées, vidéos des matchs)
- Afficher une galerie photos des activités de l'association
- Donner la possibilité aux administrateurs de mettre à jour le contenu sans avoir besoin de coder

Méthode de travail

Le projet a été réalisé seul, en autonomie complète. J'ai d'abord créé les maquettes sur Figma pour valider le design avant de commencer le développement. J'ai utilisé Git pour versionner mon code tout au long du développement et GitHub pour héberger le dépôt. Le déploiement en production est assuré par Railway, qui reconstruit et redéploie automatiquement le site à chaque fois que je pousse du code sur GitHub.

2. Technologies et outils utilisés

Front-end

React 18

React est une bibliothèque JavaScript open source créée par Meta (Facebook). Elle permet de construire des interfaces utilisateur en les découpant en petits blocs indépendants appelés composants. Chaque composant gère son propre affichage et peut être réutilisé sur plusieurs pages. J'ai utilisé React pour construire toutes les pages du site : accueil, galerie, FC18, membres, etc.

Hook useState

Un hook est une fonction spéciale fournie par React. Le hook useState permet de créer une variable d'état dans un composant : c'est une variable dont le changement de valeur provoque automatiquement un rafraîchissement de l'affichage. Par exemple, sur la page FC18, je l'utilise pour savoir quel onglet est actif (Joueurs ou Vidéos) et pour mémoriser la page courante de la pagination.

Hook useEffect

Le hook useEffect permet d'exécuter du code au moment où un composant s'affiche ou lorsque certaines valeurs changent. Je l'utilise principalement pour charger les données depuis l'API au moment où une page s'ouvre. Par exemple, quand l'utilisateur arrive sur la page FC18, useEffect déclenche automatiquement les appels à l'API pour récupérer la liste des joueurs, les trophées et les vidéos.

Hook useRef

Le hook useRef permet d'accéder directement à un élément HTML dans le code JavaScript sans passer par le système d'état de React. Sur la page des vidéos FC18, je l'utilise pour garder une référence à chaque lecteur vidéo. Cela me permet, quand l'utilisateur lance une vidéo, de mettre automatiquement en pause toutes les autres vidéos qui seraient déjà en train de jouer.

Hook personnalisé useSEO

J'ai créé mon propre hook useSEO pour gérer le référencement naturel (SEO) du site. Ce hook modifie dynamiquement le titre de la page (ce qui apparaît dans l'onglet du navigateur) et la balise meta description (le texte qui s'affiche dans les résultats Google) en fonction de la page visitée. Cela améliore la visibilité du site sur les moteurs de recherche.

React Router DOM v6

React Router est la bibliothèque qui gère la navigation entre les pages d'une application React. Sans elle, React ne saurait pas quelle page afficher selon l'URL. Je l'ai utilisé pour définir toutes

les routes du site (/accueil, /galerie, /fc18...) et pour protéger les pages d'administration avec un composant ProtectedRoute qui redirige vers la page de connexion si l'utilisateur n'est pas connecté.

Vite

Vite est un outil de développement et de construction pour les projets JavaScript modernes. Il démarre très rapidement et met à jour l'affichage dans le navigateur presque instantanément lorsqu'on modifie le code. En production, Vite compile et optimise tous les fichiers JavaScript et CSS en fichiers compressés prêts à être déployés.

Back-end

Node.js

Node.js est un environnement d'exécution qui permet de faire tourner du code JavaScript côté serveur, c'est-à-dire en dehors du navigateur. Grâce à Node.js, j'ai pu écrire le serveur du site en JavaScript, le même langage que le front-end.

Express.js

Express est un framework pour Node.js qui simplifie la création de serveurs web et d'API REST. Il permet de définir des routes (des adresses URL auxquelles le serveur répond) et de traiter les requêtes envoyées par le front-end. Par exemple, quand React demande la liste des joueurs FC18, Express reçoit cette demande, interroge la base de données MariaDB, et renvoie la liste en JSON.

MariaDB

MariaDB est un système de gestion de base de données relationnelle, compatible avec MySQL. Une base de données relationnelle organise les données sous forme de tables reliées entre elles. J'ai utilisé MariaDB pour stocker toutes les données du site : événements, photos, joueurs FC18, trophées, vidéos, membres du bureau et comptes administrateurs.

Cloudinary

Cloudinary est un service cloud spécialisé dans le stockage et la gestion des médias (images et vidéos). Au lieu de stocker les fichiers directement sur le serveur, je les envoie sur Cloudinary qui les héberge et génère des URLs d'accès. Cloudinary permet aussi de générer automatiquement des miniatures vidéo en ajoutant un paramètre dans l'URL.

JWT (JSON Web Token)

JWT est un standard pour créer des jetons d'authentification sécurisés. Quand un administrateur se connecte, le serveur génère un token JWT signé et le renvoie au front-end. Le front-end stocke ce token et l'envoie avec chaque requête vers les routes protégées. Le serveur vérifie la signature du token pour s'assurer qu'il n'a pas été falsifié.

bcrypt

bcrypt est un algorithme de hachage des mots de passe. Il serait dangereux de stocker les mots de passe en clair dans la base de données. bcrypt transforme le mot de passe en une chaîne de caractères illisible et irréversible (le hash). Lors de la connexion, bcrypt compare le mot de passe saisi avec le hash stocké pour vérifier que c'est le bon.

Nodemailer

Nodemailer est une bibliothèque Node.js qui permet d'envoyer des emails depuis le serveur. Je l'utilise pour la fonctionnalité de réinitialisation de mot de passe : quand un admin clique sur "Mot de passe oublié", le serveur génère un lien unique et Nodemailer l'envoie par email à l'administrateur.

Railway

Railway est une plateforme cloud de type PaaS (Platform as a Service) qui héberge des applications web. J'y ai déployé le serveur Express et la base de données MariaDB. À chaque fois que je fais un git push, Railway détecte automatiquement le changement et redéploie le site — c'est ce qu'on appelle le déploiement continu.

3. Architecture de l'application

L'application suit une architecture client-serveur découplée : le front-end (React) et le back-end (Express) sont deux applications indépendantes qui communiquent via une API REST. Le front-end n'accède jamais directement à la base de données : il passe toujours par le serveur Express.

API REST — Principales routes

Méthode	Route	Description	Accès
GET	/api/events	Liste des événements	Public
GET	/api/gallery	Photos de la galerie	Public
GET	/api/fc18/players	Joueurs FC18	Public
GET	/api/fc18/trophies	Trophées FC18	Public
GET	/api/fc18/videos	Vidéos des matchs	Public
POST	/api/admin/login	Connexion administrateur	Public
POST	/api/admin/events	Ajouter un événement	Admin (JWT)
DELETE	/api/admin/events/:id	Supprimer un événement	Admin (JWT)
POST	/api/admin/gallery	Ajouter une photo	Admin (JWT)
POST	/api/admin/fc18/videos	Uploader une vidéo	Admin (JWT)

4. Base de données

La base de données du projet est une base MariaDB relationnelle nommée AJBF. Elle contient 7 tables principales, chacune correspondant à une fonctionnalité du site.

Table	Description	Colonnes principales
events	Événements de l'association	id, title, description, date, image_url
gallery	Photos de la galerie	id, image_url, category, created_at
fc18_players	Joueurs de l'équipe FC18	id, image_url, created_at
fc18_trophies	Trophées remportés	id, title, year, image_url
fc18_videos	Vidéos des matchs	id, url, title, description
mandats	Membres du bureau	id, name, role, photo_url, mandat_year
admin_users	Comptes administrateurs	id, email, password_hash, reset_token

Sécurité de la base de données

Pour sécuriser l'accès aux données, j'ai créé un utilisateur MariaDB dédié nommé `ajbf_app` avec des droits limités au strict nécessaire. Ce principe s'appelle le moindre privilège : on ne donne à chaque partie du système que les droits dont elle a réellement besoin. Si la partie publique du site était compromise, l'attaquant ne pourrait pas modifier ni supprimer de données.

5. Fonctionnalités développées

Page d'accueil

La page d'accueil présente l'association avec une section hero, un résumé des derniers événements chargés dynamiquement depuis l'API, et des liens vers les différentes sections du site.

Elle est optimisée pour le référencement avec des balises meta personnalisées via le hook `useSEO`.

Galerie photos

La galerie affiche toutes les photos de l'association chargées depuis la base de données.

Elle inclut une lightbox interactive : quand l'utilisateur clique sur une photo, elle s'affiche en grand format par-dessus la page.

La galerie propose un système de pagination pour ne pas charger toutes les photos d'un coup.

Page FC18 — Équipe de Football

Cette page est l'une des plus complexes du site. Elle comporte :

Un système d'onglets pour basculer entre joueurs et vidéos sans recharger la page

Une grille de joueurs avec pagination (12 par page) et lightbox

Une grille de vidéos avec pagination (6 par page) et miniatures automatiques (Cloudinary)

Un mécanisme de lecture exclusive : une seule vidéo peut jouer à la fois

Une section trophées affichant les titres remportés

Espace administration

L'espace d'administration est accessible uniquement via l'URL /backstage, nom discret choisi pour ne pas attirer l'attention. Il comprend :

Connexion sécurisée — email + mot de passe vérifié avec bcrypt, token JWT en retour

Session avec timeout — expiration automatique après inactivité

Réinitialisation de mot de passe — lien unique envoyé par email via Nodemailer

Gestion du contenu — ajout et suppression d'événements, photos, joueurs, trophées et vidéos

Upload de fichiers — Multer reçoit les fichiers et les envoie sur Cloudinary

Pages légales et conformité

Le site intègre une page de mentions légales (informations sur l'éditeur, l'hébergeur), une politique de confidentialité (conformité RGPD sur la collecte des données) et un formulaire de contact avec adresse email de retour. Ces pages sont obligatoires pour tout site web professionnel publié en France.

6. Captures d'écran

Hook useSEO — Gestion du référencement

```
import { useEffect } from "react";

const SITE_NAME = "AJBF";
const DEFAULT_DESCRIPTION = "Association des Jeunes de Baediam et de France – Solidarité, éducation et culture";
const SITE_URL = import.meta.env.VITE_SITE_URL || "https://ajbf.fr";
const DEFAULT_IMAGE = `${SITE_URL}/og-image.jpg`;

function setMeta(name, content, attr = "name") {
  let el = document.querySelector(`meta[${attr}="${name}"]`);
  if (!el) {
    el = document.createElement("meta");
    el.setAttribute(attr, name);
    document.head.appendChild(el);
  }
  el.setAttribute("content", content);
}

function setCanonical(path) {
  let el = document.querySelector('link[rel="canonical"]');
  if (!el) {
    el = document.createElement("link");
    el.setAttribute("rel", "canonical");
    document.head.appendChild(el);
  }
  el.setAttribute("href", `${SITE_URL}${path}`);
}

export function useSEO({ title, description, path, image } = {}) {
  useEffect(() => {
    const fullTitle = title
      ? `${title} - ${SITE_NAME}`
      : `${SITE_NAME} - Association des Jeunes de Baediam et de France`;
    const desc = description || DEFAULT_DESCRIPTION;
    const img = image || DEFAULT_IMAGE;
    const url = `${SITE_URL}${path || "/"}`;

    document.title = fullTitle;
    setMeta("description", desc);

    setMeta("og:title", fullTitle, "property");
    setMeta("og:description", desc, "property");
    setMeta("og:image", img, "property");
    setCanonical(path);
  });
}
```

Ln 1, Col 1 Tab Size: 4

Génération automatique des miniatures vidéo

```
const getVideoPoster = (url) => {
  if (!url || !url.includes("cloudinary.com")) return undefined;
  return url.replace("/upload/", "/upload/so_0/").replace(/\.(\mp4)$\/i, ".jpg");
};
```

Protection des routes admin (ProtectedRoute)

```
import { Navigate } from "react-router-dom";
import { isAdminAuthenticated } from "../../lib/adminAuth";

export default function ProtectedRoute({ children }) {
  if (!isAdminAuthenticated()) {
    return <Navigate to="/backstage/login" replace />;
  }

  return children;
}
```

Route API Express — Récupération des joueurs FC18

```
const __dirname = dirname(fileURLToPath(import.meta.url));

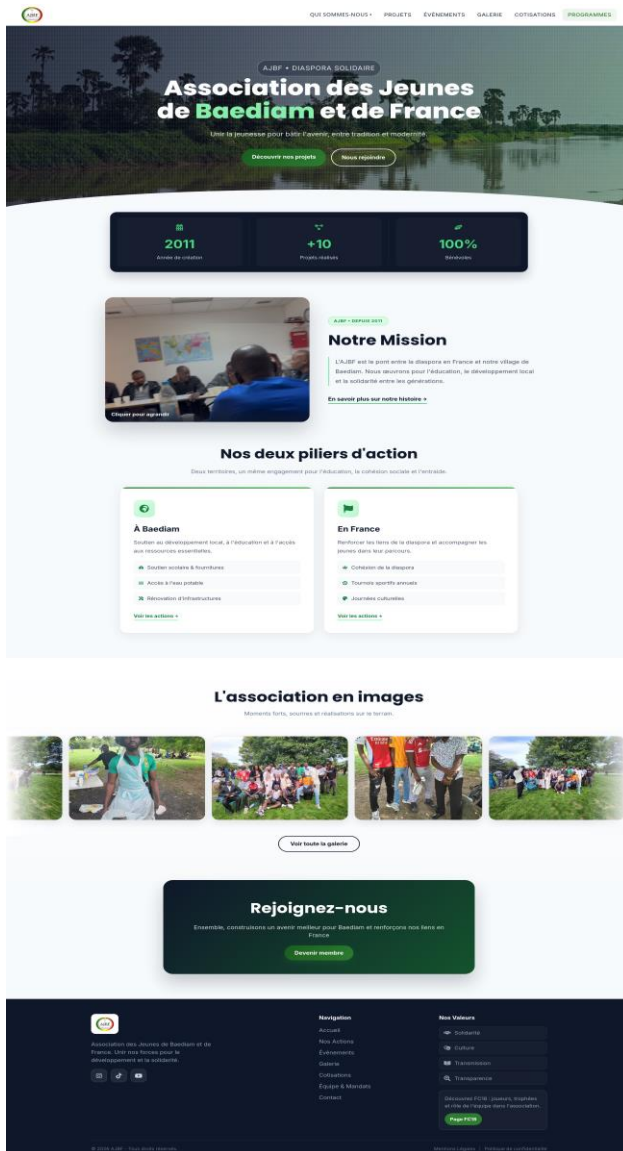
const app = express();
const port = Number(process.env.PORT) || 3000;

const allowedOrigins = (process.env.CORS_ORIGINS || "http://localhost:5173")
  .split(",")
  .map((o) => o.trim());

app.use(cors({
  origin: (origin, cb) => {
    if (!origin || allowedOrigins.includes(origin)) return cb(null, true);
    cb(null, false);
  },
  credentials: true,
}));
app.use(express.json({ limit: "10mb" }));

app.use("/api", publicRoutes);
app.use("/api/auth", authRoutes);
app.use("/api/admin", adminRoutes);
```

Captures d'écran du site en production





Trophées remportés



Nos joueurs



Précédent 1 2 Suivant

7. Difficultés rencontrées et solutions

Problème 1 — Upload de vidéos bloqué (erreur 413)

Railway limite la taille des requêtes HTTP à environ 100 MB. Or certaines vidéos des matchs dépassent cette limite (jusqu'à 240 MB). Les uploads échouaient avec une erreur 413 — Request Entity Too Large.

Solution : J'ai intégré Cloudinary pour que les vidéos soient envoyées directement depuis le navigateur vers les serveurs Cloudinary, sans passer par le serveur Railway. Le serveur ne reçoit plus le fichier lui-même, seulement l'URL une fois l'upload terminé.

Problème 2 — Vidéos sans aperçu avant lecture

Les vidéos apparaissaient avec un fond noir avant que l'utilisateur appuie sur lecture, ce qui rendait l'interface peu attrayante.

Solution : Pour les vidéos Cloudinary, j'ai créé la fonction `getVideoPoster` qui génère une miniature via la transformation d'URL Cloudinary (paramètre `so_0`). Pour les anciennes vidéos locales, j'utilise `preload="metadata"` qui demande au navigateur d'afficher la première image de la vidéo sans la télécharger entièrement.

Problème 3 — Configuration environnement dev vs production

En développement local, l'API est sur `localhost:3000`. En production, elle est sur Railway. La variable `VITE_API_BASE_URL` était mal configurée, ce qui faisait échouer les appels API en local.

Solution : En développement, je laisse `VITE_API_BASE_URL` vide, et Vite redirige automatiquement les requêtes `/api/*` vers `localhost:3000` grâce au proxy configuré dans `vite.config.js`. En production, la variable pointe vers l'URL Railway.

8. Compétences DWWW couvertes

Compétence	Ce que j'ai réalisé concrètement
CP1 — Config environnement	Configuration VS Code, Git, Node.js, variables d'environnement .env, proxy Vite pour le développement local
CP2 — Maquetter	Création des maquettes Figma pour certaines pages avant de commencer le développement
CP3 — Interfaces statiques	HTML/CSS responsive sur toutes les pages, adapté mobile et desktop, CSS séparé par composant, aucun style inline
CP4 — Partie dynamique	React 18, hooks (useState, useEffect, useRef, useSEO), consommation API REST avec fetch, routing SPA, lightbox, pagination, onglets
CP5 — Base de données	Schéma MariaDB avec 7 tables, utilisateur SQL avec droits limités (principe de moindre privilège)
CP6 — Accès aux données	Requêtes SQL via mysql2, endpoints GET/POST/DELETE sécurisés, validation des entrées côté serveur
CP7 — Composants métier	API REST Express complète, authentification JWT + bcrypt, upload Cloudinary via Multer, réinitialisation de mot de passe par email avec Nodemailer
CP8 — Documenter déploiement	Déploiement Railway avec CI/CD automatique via Git push, configuration DNS OVH pour le domaine baydjam.fr, variables d'environnement production

Projet 2 — Plateforme de festival de films IA (MarsAI)

1. Présentation du projet et contexte

MarsAI est une plateforme web dédiée à un festival de courts-métrages réalisés à l'aide de l'intelligence artificielle. Elle permet aux réalisateurs de soumettre leurs films, aux administrateurs de les examiner et de les sélectionner, et au public de les découvrir. Le projet a été réalisé en équipe de 6 développeurs : 4 développeurs front-end et 2 développeurs back-end. J'ai travaillé sur la partie back-end, en binôme avec un autre développeur.

Mon rôle consistait à concevoir et développer les routes Express et les contrôleurs CRUD : création, lecture, modification et suppression des ressources du festival (vidéos, événements, jury, sponsors, réservations...). Mon binôme back-end a pris en charge la gestion de l'authentification JWT et l'accès à la base de données via Prisma. Tout le back-end a été écrit en TypeScript, ce qui nous a permis de détecter les erreurs en amont et de garantir la cohérence des données entre les couches.

Qu'est-ce qu'un festival de courts-métrages IA ?

Un festival de courts-métrages IA est un événement où des créateurs soumettent des films de courte durée entièrement ou partiellement produits avec des outils d'intelligence artificielle (génération d'images, de sons, de scénarios). La plateforme gère le cycle complet : soumission, revue par un jury, sélection, et diffusion des films primés.

2. Technologies et outils utilisés

Les technologies listées ci-dessous sont celles utilisées par l'ensemble de l'équipe sur ce projet. Les technologies que j'ai personnellement utilisées dans ma partie (routes et contrôleurs CRUD) sont Node.js, TypeScript et Express.js. Les autres (JWT, bcrypt, Redis, AWS S3, BullMQ) ont été mises en place par mon binôme back-end.

Node.js

Node.js est un environnement d'exécution JavaScript côté serveur. Il permet d'écrire le back-end d'une application web en JavaScript (ou TypeScript), en dehors du navigateur. Il est particulièrement adapté aux APIs REST car il gère de nombreuses requêtes simultanées de façon légère et efficace.

TypeScript

TypeScript est une surcouche de JavaScript qui ajoute le typage statique. Concrètement, cela signifie que chaque variable, chaque paramètre de fonction, chaque réponse d'API est déclaré avec un type précis (string, number, objet avec telle structure...). Le compilateur détecte les erreurs avant même l'exécution du code, ce qui réduit les bugs en production.

Express.js

Express.js est le framework web le plus utilisé pour Node.js. Il simplifie la création de routes HTTP (GET, POST, PUT, DELETE), l'application de middlewares (authentification, validation, gestion des erreurs) et l'organisation du code en contrôleurs et services. C'est la colonne vertébrale de l'API MarsAI.

Prisma ORM

Prisma est un ORM (Object-Relational Mapper) : un outil qui fait le lien entre le code TypeScript et la base de données MySQL. Au lieu d'écrire des requêtes SQL à la main, on décrit les tables dans un fichier `schema.prisma`, et Prisma génère automatiquement les fonctions TypeScript pour lire, créer, modifier et supprimer les données. Cela garantit la sécurité des requêtes et évite les injections SQL.

JWT — JSON Web Token

Un JWT est un jeton numérique signé qui permet d'authentifier un utilisateur sans stocker de session côté serveur. Lors de la connexion, le serveur génère un token contenant l'identifiant et le rôle de l'administrateur, signé avec une clé secrète. À chaque requête protégée, le client envoie ce token dans les en-têtes HTTP ; le serveur le vérifie et autorise ou refuse l'accès.

bcrypt

bcrypt est une fonction de hachage spécialement conçue pour les mots de passe. Contrairement à un simple chiffrement, un mot de passe haché avec bcrypt ne peut pas être retrouvé, même si la base de données est compromise. Lors de la connexion, bcrypt compare le mot de passe saisi avec le hash stocké pour valider l'authentification.

Redis

Redis est une base de données en mémoire, utilisée ici pour deux usages : la liste noire des tokens JWT (lors d'une déconnexion, le token est ajouté à Redis pour être invalidé immédiatement) et le cache des réponses API (les données fréquemment demandées sont mises en cache pour accélérer les temps de réponse).

AWS S3

Amazon S3 (Simple Storage Service) est un service de stockage de fichiers en ligne. Dans MarsAI, les fichiers vidéo et les images de couverture soumis par les participants sont stockés sur S3 plutôt que sur le serveur local. Cela permet de gérer des fichiers volumineux sans saturer le disque du serveur, et de les distribuer rapidement à l'échelle mondiale.

BullMQ

BullMQ est une bibliothèque de gestion de files d'attente de tâches (queue). Dans MarsAI, les vidéos soumises ne sont pas traitées immédiatement — elles sont placées dans une file

d'attente BullMQ, puis un worker les traite en arrière-plan : upload sur S3, envoi sur YouTube, vérification du statut. Cela évite de bloquer le serveur pendant des opérations longues.

3. Architecture et organisation du back-end

Le back-end MarsAI suit une architecture MVC découplée (Modèle-Vue-Contrôleur), organisée en couches distinctes pour faciliter la maintenance et les tests :

- Routes (src/routes/) — définissent les endpoints HTTP et appliquent les middlewares d'authentification
- Contrôleurs (src/controllers/) — reçoivent la requête, appellent les services, renvoient la réponse JSON
- Services (src/services/) — contiennent la logique métier (upload S3, envoi email, YouTube API)
- Middlewares (src/middlewares/) — vérifient le JWT, protègent les routes admin, gèrent le cache Redis
- Prisma (prisma/schema.prisma) — définit toutes les tables et génère le client TypeScript
- BullMQ Workers (src/workers/) — traitent les tâches asynchrones (upload vidéo, YouTube)

4. Base de données

La base de données MySQL comporte 16 tables, modélisées dans le fichier schema.prisma. Chaque table correspond à une entité métier du festival :

Table	Rôle
videos	Films soumis au festival (titre, description, fichier, statut, pays...)
admins	Comptes administrateurs avec rôle (admin, super_admin)
editions	Éditions du festival (année, nom, phase en cours)
countries	Pays d'origine des films soumis
events	Événements liés au festival (ateliers, projections...)
jury	Membres du jury par édition
reviews	Critiques des juges sur chaque film (note, commentaire, statut)
reservations	Inscriptions du public aux événements
prized_videos	Films primés avec le nom du prix attribué

process_queue	File d'attente BullMQ : état du traitement de chaque vidéo
tokens	Tokens d'invitation pour créer un compte admin
newsletters	Emails abonnés à la newsletter
subtitles	Fichiers de sous-titres associés aux vidéos
sponsors	Sponsors par édition (officiel, média, technique...)
content	Contenus éditoriaux modifiables (textes du site)
settings	Paramètres globaux de la plateforme

5. Fonctionnalités développées (partie back-end)

Dans ce projet d'équipe, je me suis concentré sur la conception et le développement des routes Express et des contrôleurs CRUD. Mon binôme back-end a pris en charge l'authentification JWT, Redis, BullMQ et le système d'invitation par email.

Routes et contrôleurs CRUD

Pour chaque ressource du festival, j'ai mis en place les opérations CRUD complètes (Create, Read, Update, Delete) accessibles via l'API REST. Chaque route est reliée à un contrôleur qui reçoit la requête, interagit avec la base de données, et renvoie une réponse JSON. Voici les principales routes développées :

Méthode	Route	Description
GET	/api/videos	Liste paginée des films (avec filtres)
POST	/api/videos	Soumission d'un nouveau film
PUT	/api/videos/:id	Modifier le statut ou les infos d'un film
DELETE	/api/videos/:id	Supprimer un film
GET	/api/events	Liste des événements du festival
POST	/api/events	Créer un événement
POST	/api/reservations	Inscrire un participant à un événement
GET	/api/jury	Liste des membres du jury par édition
POST	/api/reviews	Soumettre une critique sur un film
GET	/api/sponsors	Liste des sponsors par édition

Organisation des contrôleurs

Chaque ressource dispose de son propre fichier contrôleur dans `src/controllers/`. Par exemple, `video.controller.js` gère toutes les opérations sur les films : récupération avec pagination, filtrage par statut ou par édition, création avec validation des champs obligatoires, mise à jour et suppression. Cette organisation permet de garder le code lisible et de localiser rapidement chaque fonctionnalité.

6. Difficultés rencontrées et solutions

Difficulté 1 — Ajustements des routes au retour des développeurs front-end

Chaque fois que je publiais une nouvelle route Express, les développeurs front-end l'intégraient et revenaient avec des retours : un champ manquant dans la réponse JSON, un format de date incorrect, ou une ressource imbriquée attendue mais absente. Par exemple, pour la route GET /api/videos, le front avait besoin que chaque vidéo inclue le nom du pays et de l'édition, pas seulement leurs identifiants. J'ai dû modifier le contrôleur pour inclure ces relations dans la requête Prisma. Ce cycle de retours m'a appris à anticiper les besoins du front et à mieux structurer les réponses API dès le départ.

Difficulté 2 — Comprendre et appliquer la structure MVC du projet

Le projet suivait une architecture MVC stricte avec des couches séparées (routes, contrôleurs, services). Au début, je ne savais pas toujours où placer la logique : dans la route, dans le contrôleur ou dans un service ? J'ai analysé le code existant de mon binôme et les conventions du projet pour comprendre cette séparation. Cela m'a permis d'écrire des contrôleurs propres, qui se contentent de recevoir la requête et de renvoyer la réponse, sans logique métier à l'intérieur.

Difficulté 3 — Travailler en équipe avec Git sur un même dépôt

Six développeurs travaillaient simultanément sur le même dépôt Git. Des conflits de fusion (merge conflicts) survenaient régulièrement, notamment sur les fichiers de routes et les fichiers de configuration. J'ai appris à créer des branches dédiées par fonctionnalité, à faire des pull requests et à résoudre les conflits manuellement avant de fusionner, ce qui a rendu la collaboration plus fluide au fil du projet.

7. Compétences DWWM couvertes

Compétence	Ce qui a été fait dans MarsAI
CP5 — BDD relationnelle	Compréhension et utilisation d'une BDD MySQL avec 16 tables, relations et clés étrangères modélisées dans Prisma
CP6 — Composants accès données	Accès aux données via Prisma ORM dans les contrôleurs : lecture paginée, filtres, création et suppression
CP7 — Composants métier serveur	Développement des routes Express et contrôleurs CRUD pour videos, events, jury, sponsors, reservations, reviews
CP8 — Documenter et déployer	Participation à la documentation Swagger/OpenAPI des endpoints développés, travail avec variables

d'environnement

Projet 3 — API [Cahier de Recettes]

1. Présentation du projet et contexte

Le Cahier de Recettes est une API REST back-end développée seul, en autonomie complète, dans le cadre de ma formation.

L'objectif était de concevoir et réaliser une application de gestion de recettes de cuisine en ligne, permettant à des utilisateurs inscrits d'ajouter, consulter, modifier et supprimer leurs recettes, ainsi que de laisser des commentaires.

Ce projet m'a permis de travailler pour la première fois avec une base de données NoSQL (MongoDB) au lieu d'une base relationnelle classique, et d'approfondir la mise en place d'une API sécurisée avec authentification JWT, documentation Swagger et architecture MVC en Node.js.

Objectifs du projet

Créer une API REST complète pour gérer des recettes (CRUD complet)

Mettre en place un système d'inscription et de connexion sécurisé (JWT + bcrypt)

Permettre aux utilisateurs authentifiés de commenter les recettes

Utiliser MongoDB comme base de données NoSQL orientée documents

Documenter toutes les routes avec Swagger/OpenAPI

Méthode de travail

Le projet a été réalisé seul, J'ai structuré le code selon le pattern MVC (Modèle-Vue-Contrôleur) dès le départ, en séparant clairement les modèles Mongoose, les contrôleurs, les routes et les middlewares. Git a été utilisé pour versionner le code, avec le dépôt hébergé sur GitHub.

2. Technologies et outils utilisés

Node.js

Node.js est l'environnement d'exécution JavaScript côté serveur. Il permet d'écrire le back-end d'une application web en JavaScript, en dehors du navigateur. J'ai utilisé Node.js comme base du serveur de l'API.

Express.js

Express.js est le framework Node.js utilisé pour définir les routes HTTP (GET, POST, PUT, DELETE), appliquer les middlewares (authentification, gestion d'erreurs) et renvoyer les réponses JSON. C'est la colonne vertébrale de l'API.

MongoDB

MongoDB est une base de données NoSQL orientée documents. Contrairement aux bases SQL (MySQL, MariaDB) qui stockent les données dans des tables avec des colonnes fixes, MongoDB stocke les données sous forme de documents JSON flexibles (BSON). Cela permet d'avoir des structures de données différentes d'un document à l'autre. J'ai utilisé MongoDB pour stocker les utilisateurs, les recettes et les commentaires.

Mongoose

Mongoose est une bibliothèque Node.js qui facilite l'utilisation de MongoDB. Elle permet de définir des schémas (la structure attendue des documents) et des modèles (les objets TypeScript/JavaScript correspondant aux collections). Mongoose gère aussi la validation des données avant l'insertion en base.

JWT — JSON Web Token

JWT est le standard d'authentification utilisé pour sécuriser les routes de l'API. Lors de la connexion, le serveur génère un token signé contenant l'identifiant de l'utilisateur. Ce token est envoyé dans les en-têtes HTTP de chaque requête protégée et vérifié par le middleware d'authentification.

bcrypt

bcrypt est l'algorithme de hachage utilisé pour les mots de passe. Les mots de passe ne sont jamais stockés en clair dans MongoDB : bcrypt les transforme en une chaîne irréversible. Lors de la connexion, bcrypt compare le mot de passe saisi avec le hash stocké.

Swagger / OpenAPI

Swagger est un outil de documentation automatique d'API REST. J'ai créé un fichier `swagger.json` décrivant toutes les routes, leurs paramètres et leurs réponses. La documentation interactive est accessible à l'URL `/api-docs`, ce qui permet de tester l'API directement depuis un navigateur.

3. Architecture de l'application

L'application suit le pattern MVC (Modèle-Vue-Contrôleur) avec une séparation claire des responsabilités :

`models/` — Schémas Mongoose (User, Recette, Commentaire) : définissent la structure des documents MongoDB

`controllers/` — Logique métier : reçoivent la requête, interagissent avec MongoDB, renvoient la réponse JSON

`routes/` — Définition des endpoints HTTP et application des middlewares d'authentification

`middlewares/` — Middleware d'authentification JWT : vérifie le token sur les routes protégées

app.js — Configuration Express, connexion MongoDB, enregistrement des routes

server.js — Point d'entrée : démarre le serveur sur le port défini

swagger.json — Documentation complète de toutes les routes de l'API

Routes API principales

Méthode	Route	Description
POST	/users/register	Créer un compte utilisateur
POST	/users/login	Connexion — retourne un token JWT
GET	/users	Liste tous les utilisateurs (protégé)
GET	/recettes	Liste toutes les recettes (public)
POST	/recettes	Créer une recette (protégé)
GET	/recettes/:id	Détail d'une recette (public)
PUT	/recettes/:id	Modifier une recette (protégé)
DELETE	/recettes/:id	Supprimer une recette (protégé)
GET	/recettes/:id/commentaires	Lister les commentaires (public)
POST	/recettes/:id/commentaires	Ajouter un commentaire (protégé)
DELETE	/commentaires/:id	Supprimer un commentaire (protégé)

4. Base de données — MongoDB (NoSQL)

La base de données MongoDB contient 3 collections, chacune correspondant à un modèle Mongoose :

Collection	Rôle	Champs principaux
users	Comptes utilisateurs	name, email, password (haché), createdAt, updatedAt
recettes	Recettes de cuisine	titre, ingredients[], etapes[], auteur, date
commentaires	Commentaires sur les recettes	contenu, auteur, recette (référence), date

SQL vs NoSQL — différence clé

Dans une base SQL (Projet 1 et 2), les données sont organisées en tableaux rigides avec des colonnes fixes. Dans MongoDB, chaque document est un objet JSON indépendant : deux recettes peuvent avoir des structures légèrement différentes sans que cela pose problème. MongoDB est particulièrement adapté aux données dont la structure peut évoluer (ajouter un champ "temps_preparation" à certaines recettes sans modifier toutes les autres).

5. Fonctionnalités développées

Authentification utilisateur

Tout utilisateur peut créer un compte en fournissant un nom, un email et un mot de passe. Le mot de passe est haché avec bcrypt avant d'être stocké dans MongoDB. Lors de la connexion, le serveur vérifie le mot de passe avec bcrypt et génère un token JWT signé avec une clé secrète. Ce token doit être envoyé dans les en-têtes de toutes les requêtes qui modifient des données.

CRUD Recettes

Les utilisateurs authentifiés peuvent créer des recettes en fournissant un titre, une liste d'ingrédients et une liste d'étapes de préparation. Les recettes sont lisibles publiquement (sans authentification). Seul l'auteur d'une recette peut la modifier ou la supprimer.

Commentaires

N'importe quel utilisateur connecté peut commenter une recette. Les commentaires sont liés à une recette via une référence MongoDB (l'identifiant unique de la recette). Cette relation entre collections est l'équivalent NoSQL d'une clé étrangère en SQL.

Documentation Swagger

L'ensemble des routes de l'API est documenté dans le fichier swagger.json. La documentation interactive est accessible à l'adresse /api-docs et permet de visualiser toutes les routes, leurs paramètres, les réponses attendues, et de tester directement les appels API depuis le navigateur. C'est une bonne pratique professionnelle pour permettre aux développeurs front-end ou aux équipes tierces de comprendre et d'utiliser l'API.

6. Difficultés rencontrées et solutions

Difficulté 1 — Passer du SQL au NoSQL : penser en documents

Habitué aux bases SQL avec des tables et des jointures, j'ai eu du mal à modéliser les relations en MongoDB. Par exemple, pour relier un commentaire à une recette, en SQL on utilise une clé étrangère (recette_id). En MongoDB, on stocke l'identifiant de la recette directement dans le document du commentaire. J'ai étudié la documentation Mongoose et pratiqué avant de trouver la bonne approche.

Difficulté 2 — Sécuriser les routes sans répéter du code

Au début, je vérifiais le token JWT manuellement dans chaque contrôleur, ce qui dupliquait du code. J'ai refactorisé en créant un middleware d'authentification dédié (authMiddleware) qui vérifie le token une seule fois et passe l'identifiant de l'utilisateur à la suite de la requête. Toute route protégée utilise ce middleware.

Difficulté 3 — Configurer Swagger pour documenter correctement l'API

La configuration de Swagger avec swagger-ui-express nécessite de décrire chaque route dans le fichier swagger.json selon le standard OpenAPI 3.0. J'ai appris à structurer ce fichier en lisant la documentation officielle et en regardant des exemples, ce qui m'a permis de produire une documentation claire et utilisable.

7. Compétences DWWM couvertes

Compétence	Ce que j'ai réalisé concrètement
CP1 — Config environnement	Mise en place de l'environnement Node.js, configuration du fichier .env (port, URI MongoDB, secret JWT), gestion des dépendances avec npm
CP5 — Base de données	Conception et utilisation d'une base de données NoSQL MongoDB avec 3 collections, schémas Mongoose avec validation des données
CP6 — Accès aux données SQL et NoSQL	Accès aux données via Mongoose ORM : opérations CRUD sur collections MongoDB, relations entre documents (référence recette → commentaires) — couvre la partie NoSQL du référentiel
CP7 — Composants métier serveur	API REST Express complète avec authentification JWT + bcrypt, middleware de protection des routes, CRUD recettes/commentaires/utilisateurs
CP8 — Documenter et déployer	Documentation complète de l'API avec Swagger/OpenAPI accessible sur /api-docs, fichier README avec instructions d'installation